

Documentation technique finale - GamesUP

Ce document décrit les choix réalisés sur le projet GamesUP. J'explique d'abord le cheminement de la réalisation du projet amenant la refonte de l'application. Ensuite, je détaille les principes SOLID ainsi que les bonnes pratiques de programmation mis en place .

1/ Cheminement de réalisation du projet

1.1 Limites de l'application initiale et objectifs de refonte

La refonte devait d'abord répondre aux limites structurelles de l'application mis en place par le stagiaire.

Dans l'ancien code, la couche web combine le transport HTTP et l'accès à la base de données. Le code JDBC se trouve directement dans le contrôleur et contient des identifiants de connexion codés en dur. Le code ne sépare pas clairement l'exposition de l'API, la logique métier et la persistance. La modélisation métier reste incomplète, avec des classes vides ou très peu structurées, sans contraintes explicites et sans contrats d'échange stables.

Un exemple typique se trouve dans le fichier **GameController.java**. Dans ce fichier, le contrôleur ouvre la connexion SQL et exécute les requêtes lui-même. Le fichier **Wishlist.java** montre un modèle presque vide, difficile à exploiter. Ces défauts de conception peuvent provoquer de graves problèmes de fiabilité et de sécurité. De plus, ils rendent la maintenance difficile.

Pour améliorer l'application, j'ai fixé les priorités suivantes :

- Reconstruire une architecture en couches pour que l'architecture reste facile à maintenir
- Renforcer la sécurité
- Rendre les flux CRUD fiables.
- Gérer les contrats en utilisant les DTO
- Intégrer le moteur de recommandation Python sans créer de dépendances fortes.

Le but n'était pas seulement de réparer les défauts techniques mais aussi de placer l'application dans un environnement où l'évolution, les tests et la stabilité en production sont prévisibles.

1.2 Démarche de reprise et bonnes/mauvaises pratiques

La reprise a été faite par petites étapes pour limiter le risque de régression et garder un rythme de validation constant.

Relire le besoin et examiner le code existant m'a permis de repérer les parties les plus critiques. Après le diagnostic, créer les différents diagrammes m'a aidé à visualiser l'objectif et à poser le cadre de conception.

Ensuite, j'ai réorganisé l'API Spring en couches claires (web, service, repository, domain) puis j'ai stabilisé la persistance avec Hibernate/JPA.

J'ai poursuivi par l'installation de la sécurité avec Spring Security, JWT, la gestion des rôles et le contrôle d'accès. Ensuite, j'ai construit les fonctionnalités CRUD et les DTO progressivement, en vérifiant chaque endpoint. Puis, j'ai mis en place des tests unitaires, d'intégration et de non régression sur la partie Spring.

Enfin, j'ai réalisé le travail de refonte de l'API Python de recommandation via FastAPI et créé les nouvelles routes dans Spring. Le travail a été consolidé en rédigeant la documentation technique finale.

Concernant les **bonnes pratiques de réalisation**, l'élaboration et le respect de ce plan logique m'ont permis de progresser efficacement. J'ai aussi décidé de créer une branche git dédiée pour chaque grande partie. Cette méthode m'a aidée à rester concentrée sur les tâches en cours et à être sûre d'avoir fini chaque tâche avant de "merger" avec le main.

J'ai essayé d'appliquer toutes les bonnes pratiques de programmation que je connais et je les détaille dans la partie suivante. Concernant les mauvaises pratiques, il en existe peut-être dont je ne connais pas l'existence. Si c'est le cas et que je les connaissais, j'aurai travaillé de sorte à les corriger.

Concernant l'application, celle-ci reste perfectible sur plusieurs axes concrets. La mise en place d'un pipeline de CI/CD n'était pas demandé dans le cadre de ce travail mais il serait très judicieux à mettre en place dans un contexte réel afin d'industrialiser les mises en production et homogénéiser les contrôles de qualité. Par ailleurs, l'API Python n'est pas testée, ce qui constituerait aussi un point prioritaire dans un contexte réel.

2. Respect des principes SOLID et bonnes pratiques

La conception de la nouvelle architecture a été réalisée en respectant les différents **principes SOLID**.

Le **principe de responsabilité unique (S)** est visible dans la séparation des couches : les controllers se concentrent sur le transport HTTP, les services sur les règles métier, les repositories sur l'accès aux données et les DTO sur le contrat d'échange. Un exemple direct est RecommendationController, qui reçoit la requête puis délègue le traitement à RecommendationService sans embarquer de logique de calcul de recommandations.

Le **principe d'ouverture/fermeture (O)** est respecté : les classes sont ouvertes à l'extension mais fermées à la modification. Cela facilite la maintenabilité, car de nouvelles logiques métiers peuvent être ajoutées ou faire évoluer le comportement du système sans nécessiter de modification des controllers.

Le **principe de substitution de Liskov (L)** apparaît dans la façon dont le controller dépend d'un type abstrait et non d'une implémentation concrète. En pratique, RecommendationController dépend de RecommendationService; toute implémentation respectant ce contrat peut être injectée avec les mêmes signatures et la même sémantique fonctionnelle.

Le **principe de ségrégation des interfaces (I)** est respecté par un découpage fonctionnel net. Ainsi, les interfaces sont spécialisées. Par exemple, AuthService regroupe uniquement les besoins d'authentification (login, register, refresh, logout), alors que RecommendationService se limite à l'entraînement et à la prédiction. Ainsi, un composant qui ne traite que la recommandation n'est pas forcé d'importer des méthodes liées à l'authentification.

Enfin, le **principe d'inversion des dépendances (D)** est appliqué via l'injection par constructeur. Par exemple, AuthController dépend de l'abstraction AuthService, et AuthServiceImpl reçoit ses dépendances (UserAccountRepository, JwtTokenService, RefreshTokenService) par configuration Spring, sans création manuelle d'objets dans la logique métier.

Concernant les bonnes pratiques, j'ai surtout cherché à compléter les principes SOLID avec des choix orientés qualité d'exploitation. La gestion des erreurs est centralisée pour garantir des réponses API homogènes. Les opérations métier sont encadrées par des transactions afin de fiabiliser les écritures et de limiter les incohérences de données. La configuration sensible (base de données, clés JWT, URL de l'API Python) est externalisée pour éviter les secrets en dur et faciliter le déploiement. Un mode dégradé est également prévu pour maintenir un service de recommandation utilisable lorsque le moteur ML est indisponible.

Enfin, j'ai ajouté des commentaires ciblés dans le code pour améliorer la lisibilité des parties les plus techniques. J'ai aussi ajouté des commentaires dans le code afin de faciliter sa compréhension et sa maintenance.

3. Rapport de couverture de tests

Mesure réalisée via JaCoCo à partir des rapports locaux.

Les mesures globales indiquent une couverture de 85.90 % sur les instructions, 86.10 % sur les lignes et 69.00 % sur les branches. Au total, 49 tests ont été exécutés, sans échec, sans erreur et sans test ignoré.

gamesUP												
gamesUP												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.gamesUP.gamesUP.service.impl		70%		73%	48	94	48	217	31	62	0	7
com.gamesUP.gamesUP.domain		77%		50%	33	128	47	190	31	126	0	15
com.gamesUP.gamesUP.web		95%		60%	9	64	8	217	5	59	0	8
com.gamesUP.gamesUP.security		97%		63%	8	43	3	129	0	32	0	6
com.gamesUP.gamesUP		37%		n/a	1	2	2	3	1	2	0	1
com.gamesUP.gamesUP.web.dto		100%		n/a	0	19	0	19	0	19	0	19
com.gamesUP.gamesUP.service		100%		n/a	0	1	0	2	0	1	0	1
Total	445 of 3,157	85%	31 of 100	69%	99	351	108	777	68	301	0	57

4. Travail effectué pour le système de recommandation

4.1 API Python FastAPI

L'API Python FastAPI expose trois endpoints principaux :

- POST /recommendations/train
- POST /recommendations/train/csv
- POST /recommendations/predict.

L'entraînement repose sur **NearestNeighbors** de **scikit-learn** avec une distance cosinus. Le vecteur de caractéristiques est construit à partir du prix, de la note moyenne, de l'éditeur, des catégories et des auteurs. Les variables numériques sont standardisées, les variables catégorielles sont encodées en one-hot et en multi-label, puis les artefacts de calcul (modèle, encodeurs, scaler et métadonnées) sont persistés via joblib.

Lors de l'inférence, le moteur parcourt l'historique d'achat utilisateur, recherche les "voisins" proches des jeux déjà achetés, agrège les similarités avec une pondération par rating et exclut les jeux déjà possédés. Deux comportements de repli sont prévus : en l'absence d'historique, le moteur propose une liste "top rated", si aucun score exploitable n'est obtenu, il revient également vers un fallback "top rated" en excluant les jeux déjà connus.

4.2 Intégration Spring vers Python

Du côté Spring, l'entraînement est déclenché via **POST** `/api/v1/commerce/recommendations/train` et la prédiction utilisateur via **GET** `/api/v1/commerce/recommendations/users/{userId}`. La communication inter-service repose sur RestTemplate. Une fois la réponse de prédiction reçue, le backend Spring enrichit les recommandations avec les données du catalogue afin de fournir au front une réponse métier directement exploitable.

Le choix d'introduire une logique de type RecommendationGateway (passerelle entre métier Spring et API Python) répond à un objectif de découplage. Concrètement, cette passerelle encapsule le protocole d'appel HTTP, la construction des payloads et la gestion des erreurs réseau afin que la couche métier conserve une responsabilité fonctionnelle. Cette approche permet de faire évoluer plus facilement l'infrastructure d'appel (URL, client HTTP, timeout, format de payload) sans réécrire les règles métier de recommandation. Elle facilite aussi les tests, car le point d'intégration externe peut être isolé et simulé.

Exemple concret : la couche métier exprime simplement les besoins `trainModel` et `recommendForUser`, tandis que la passerelle gère les détails d'intégration vers FastAPI (`/recommendations/train`, `/recommendations/predict`), la sérialisation des payloads et les erreurs de communication. Ainsi, un changement de client HTTP ou de configuration réseau n'impose pas de modifier les règles métier de recommandation.

4.3 Résilience appliquée

J'ai essayé de mettre en place une résilience surtout dans le contexte actuel où les données actuels sont peu/pas exploitables pour réaliser un entraînement efficace. Si l'API Python est indisponible, le backend Spring bascule vers un fallback top ventes. Si ce résultat reste insuffisant, il complète la réponse avec un fallback top notes. Cette stratégie permet de maintenir une réponse fonctionnelle, même lorsque le moteur ML n'est pas disponible.